

Frontend Interview Series 21/600: React Component Lifecycle

Question: Explain the React component lifecycle in both class components and functional components with hooks.

Explanation: React components go through a series of phases during their lifetime, from initialization to removal from the DOM. Understanding these phases is crucial for properly managing side effects, data fetching, and cleanup.

Class Component Lifecycle: The lifecycle of class components is divided into three main phases:

1. Mounting Phase:

- `constructor()`: Initialize state, bind methods
- `static getDerivedStateFromProps()`: Update state based on props
- `render()`: Create the component's UI
- `componentDidMount()`: Run after component is inserted into DOM

2. Updating Phase:

- `static getDerivedStateFromProps()`
- `shouldComponentUpdate()`: Decide if component should re-render
- `render()`
- `getSnapshotBeforeUpdate()`: Capture information before update
- `componentDidUpdate()`: Run after component updates

3. Unmounting Phase:

- `componentWillUnmount()`: Clean up before component is removed

Functional Component Lifecycle with Hooks: Hooks don't follow the same lifecycle model, but they accomplish similar goals:

- `useState`: Initializes and manages state (similar to constructor and `setState`)
- `useEffect`: Handles side effects at various points in the lifecycle
- `useLayoutEffect`: Similar to `useEffect` but fires synchronously after DOM mutations

Code Example:

Class Component with lifecycle methods

```
class ClassLifecycle extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    console.log('Constructor: Initialize component');
  }

  static getDerivedStateFromProps(props, state) {
    console.log('getDerivedStateFromProps: Before render');
    return null; // Return new state or null
  }

  componentDidMount() {
    console.log('componentDidMount: After first render');
    // Good place for API calls
  }

  shouldComponentUpdate(nextProps, nextState) {
    console.log('shouldComponentUpdate: Decide to re-render');
    return true; // Return false to prevent update
  }

  getSnapshotBeforeUpdate(prevProps, prevState) {
    console.log('getSnapshotBeforeUpdate: Before DOM update');
    return null; // Return value passed to componentDidUpdate
  }

  componentDidUpdate(prevProps, prevState, snapshot) {
    console.log('componentDidUpdate: After re-render');
    // Good place to update DOM based on state changes
  }

  componentWillUnmount() {
    console.log('componentWillUnmount: Before removal');
    // Cleanup operations: Clear timers, cancel requests
  }

  render() {
    console.log('render: Create component UI');
  }
}
```

```

    return <div>Count: {this.state.count}</div>;
  }
}

```

Functional Component with hooks

```

function HooksLifecycle() {
  console.log('Function body: Runs on every render');

  // Similar to constructor and state initialization
  const [count, setCount] = useState(0);

  // Similar to componentDidMount + componentDidUpdate +
  // componentWillUnmount
  useEffect(() => {
    console.log('useEffect (no deps): After every render');

    return () => {
      console.log('useEffect cleanup: Before next effect or unmount');
    };
  });

  // Similar to componentDidMount + componentWillUnmount
  useEffect(() => {
    console.log('useEffect (empty deps): After first render (mount)');

    return () => {
      console.log('useEffect cleanup: Before unmount');
    };
  }, []);

  // Similar to componentDidUpdate for specific props/state
  useEffect(() => {
    console.log('useEffect (with deps): When count changes');

    return () => {
      console.log('useEffect cleanup: Before next count effect');
    };
  }, [count]);

  // Similar to getSnapshotBeforeUpdate + componentDidUpdate
  useEffect(() => {

```

```

    console.log('useLayoutEffect: Synchronously after DOM mutations');

    return () => {
      console.log('useLayoutEffect cleanup');
    };
  });

  return <div>Count: {count}</div>;
}

```

Common use cases for lifecycle methods/hooks:

1. Data Fetching:

```

// Class
componentDidMount() {
  fetch('/api/data').then(res => this.setState({ data: res.data }));
}

// Hooks
useEffect(() => {
  fetch('/api/data').then(res => setData(res.data));
}, []); // Empty deps = componentDidMount equivalent

```

2. Subscriptions/Event Listeners:

```

// Class
componentDidMount() {
  window.addEventListener('resize', this.handleResize);
}
componentWillUnmount() {
  window.removeEventListener('resize', this.handleResize);
}

// Hooks
useEffect(() => {
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);

```

3. Responding to Props/State Changes:

```
// Class
componentDidUpdate(prevProps) {
  if (prevProps.userId !== this.props.userId) {
    this.fetchUserData(this.props.userId);
  }
}

// Hooks
useEffect(() => {
  fetchUserData(userId);
}, [userId]); // Runs when userId changes
```

Understanding the component lifecycle helps you manage side effects properly, optimize performance, and prevent memory leaks. With hooks, the mental model changes from "lifecycle methods" to "synchronizing effects with state/props".

Follow for more frontend interview preparation content!

[Sumedh Patil](#)